Deployment Pipeline Documentation

Current State, Proposed Solution & Recommendations

flowtali-fe | Frontend Engineering Team
Version Control: Bitbucket

Prepared: May 2026

Table of Contents

1. Current Pipeline — Problem Statement
- 1.1 Current Workflow
- 1.2 The Core Problem

2. Proposed Solution: Two-Stage Pipeline
- 2.1 Stage 1 — Preview Environment
- 2.2 Stage 2 — Integration-Dev
- 2.3 Benefits
- 2.4 Known Risks & Flaws
- 2.5 Mitigation

3. Merge Queues Explained
- 3.1 What Is a Merge Queue?
- 3.2 Bitbucket Support
- 3.3 How It Works
- 3.4 How to Enable in Bitbucket

4. Recommended Combined Approach
- 4.1 The Recommended Flow
- 4.2 Benefits Summary

5. Implementation Steps (Bitbucket)

6. Process Comparison Summary

SECTION 1

Current Pipeline — Problem Statement

1.1 Current Workflow

The existing deployment pipeline follows a straightforward linear flow. A developer creates a feature branch, completes their work, and raises a Pull Request (PR) targeting the **integration-dev** branch. This PR creation automatically triggers a deployment to a shared PR/preview environment. The team then tests the feature in that environment and, once approved, merges the PR into integration-dev.

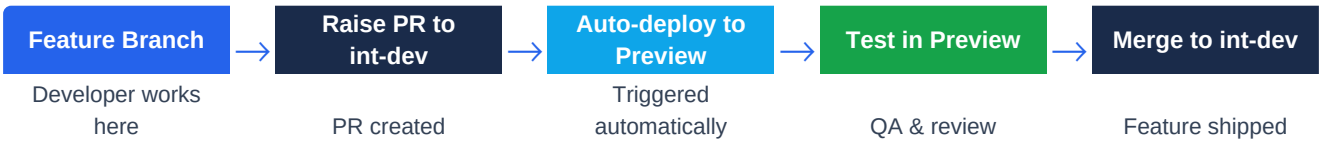

Figure 1 — Current Pipeline Flow

1.2 The Core Problem

The current model breaks down when multiple developers work concurrently. Because all PRs target the same **integration-dev** branch, the following chain of failures occurs:

- Multiple PRs are raised simultaneously against integration-dev.
- PRs conflict with each other at the branch level before any of them can be merged.
- The merge conflict prevents the auto-deploy pipeline from triggering correctly.
- The PR/preview environment either fails to deploy or reflects a broken intermediate state.
- Developers are blocked — they cannot test their work and must wait for others to resolve conflicts first.

Impact

- Developer velocity drops as teams wait on conflict resolution.
- Preview environments reflect inconsistent or broken states.
- Risk of bugs reaching integration-dev increases.
- CI/CD pipeline reliability degrades over time.

SECTION 2

Proposed Solution: Two-Stage Pipeline

To address the conflict problem, the proposal is to introduce a dedicated, standalone **preview** environment that acts as an isolated testing stage before code is merged into integration-dev. This creates a two-stage pipeline.

2.1 Stage 1 — Preview Environment

The developer raises a PR against the **preview** branch (rather than integration-dev). This triggers an auto-deploy to the preview environment. The feature is tested in isolation. Once confirmed working, the PR is merged into preview.

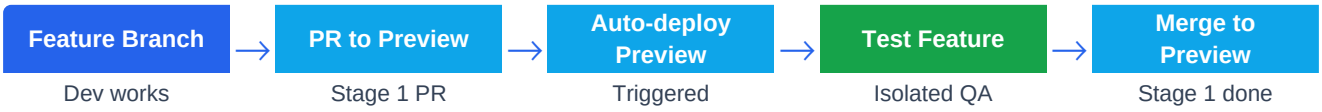

Figure 2 — Stage 1: Preview Pipeline

2.2 Stage 2 — Integration-Dev

Once the feature is fully confirmed in the preview environment, the developer raises a second PR, this time targeting **integration-dev**. The merge to integration-dev now represents verified, tested code.

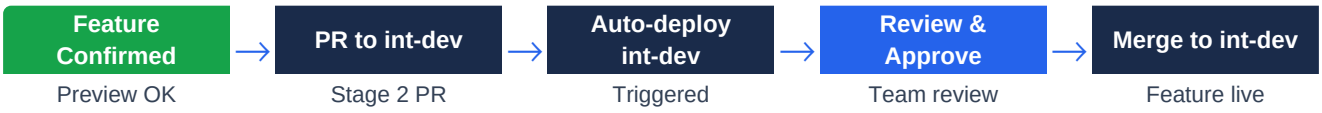

Figure 3 — Stage 2: Integration-Dev Pipeline

2.3 Benefits

- Feature testing is decoupled from the integration branch.
- integration-dev receives only verified, tested code, reducing broken states.
- Developers can work in parallel without immediately conflicting on integration-dev.
- Clear separation of concerns: preview = testing, integration-dev = integration.

2.4 Known Risks & Flaws

Important: This approach has structural risks that must be addressed

- If preview is a shared branch, merge conflicts simply relocate there — the root problem is not solved.
- Double PR overhead per feature adds process burden and slows developer workflow.
- The preview branch requires a clear lifecycle policy: when is it reset? who owns it?
- Integration issues between features are discovered later (at Stage 2), not earlier.

2.5 Mitigation

The critical mitigation for this approach is to make the preview environment **ephemeral and per-PR** rather than a shared branch. Each PR gets its own isolated preview environment that is created automatically on

PR open and destroyed on PR close or merge. This eliminates shared-branch conflict risk while preserving isolation.

SECTION 3

Merge Queues Explained

3.1 What Is a Merge Queue?

A merge queue is a mechanism that **serializes the merging of Pull Requests**. Instead of allowing multiple PRs to merge simultaneously (causing race conditions and conflicts), a merge queue processes one PR at a time. Before each merge, the queue automatically rebases the PR against the very latest state of the target branch and re-runs the CI pipeline. If the build passes, the PR is merged. If it fails, the developer is notified and the next PR in the queue continues unblocked.

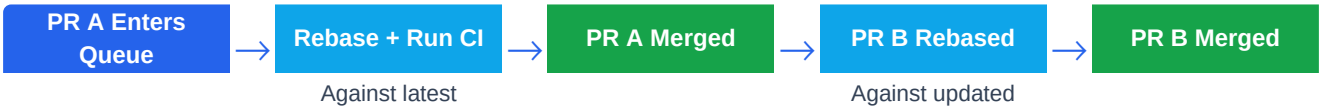

Figure 4 — Merge Queue Flow

3.2 Bitbucket Support

Bitbucket Cloud — Premium	YES	Native merge queue, configurable per branch
Bitbucket Cloud — Free / Standard	NO	Not available on lower-tier plans
Bitbucket Data Center / Server	YES	Available via branch permissions configuration

3.3 How It Works

- Developer raises a PR and marks it ready for merge.
- The PR enters the merge queue — it does not merge immediately.
- The queue picks up the PR, rebases it against the latest state of the target branch.
- CI pipeline runs against the rebased code.
- If CI passes: PR is automatically merged.
- If CI fails: developer is notified, the next PR in the queue continues.
- No two PRs merge simultaneously — race conditions are eliminated entirely.

3.4 How to Enable in Bitbucket

Step 1: Navigate to Repository Settings in your Bitbucket repo.

Step 2: Select Branch restrictions (or Merge checks, depending on plan).

Step 3: Target The integration-dev branch.

Step 4: Enable Merge queue for that branch.

Step 5: Configure Queue settings: batch size, timeout, required CI checks.

Step 6: Communicate Update the team on the new merge workflow.

SECTION 4

Recommended Combined Approach

The recommended approach combines the two-stage pipeline with Bitbucket merge queues to address both the isolation problem and the race-condition problem simultaneously.

4.1 The Recommended Flow

Stage 1 — Ephemeral Preview (per PR):

- Developer raises PR from feature branch.
- An ephemeral preview environment is automatically created for that specific PR.
- The feature is deployed and tested in complete isolation — no shared branch conflicts.
- On PR close or merge, the ephemeral environment is automatically destroyed.

Stage 2 — Merge Queue to Integration-Dev:

- Developer raises a second PR to integration-dev once the feature is confirmed.
- The PR enters the Bitbucket merge queue — no immediate merge.
- Queue rebases the PR against the latest integration-dev and runs CI.
- On CI pass: PR merges cleanly. On CI fail: developer is notified.
- No two PRs can conflict during merge — serialization is enforced by the queue.

Figure 5 — Recommended Combined Pipeline

4.2 Benefits Summary

Merge conflicts on integration-dev	Merge queue serializes all merges — no simultaneous merges possible
Isolated feature testing	Ephemeral per-PR preview environments — no shared branch pollution
Broken integration-dev state	Queue ensures only CI-passing code is ever merged
Developer visibility & status	Queue dashboard shows position, status, and CI results per PR
Shared preview branch conflicts	Eliminated — each PR has its own isolated environment
Late discovery of integration bugs	Stage 2 PR surfaces integration issues before they reach production

SECTION 5

Implementation Steps (Bitbucket)

The following steps outline how to implement the recommended combined approach within a Bitbucket-hosted repository.

1 Enable Merge Queue on integration-dev

- Go to Repository Settings → Branch restrictions.
- Select the integration-dev branch.
- Enable the Merge queue option (requires Bitbucket Cloud Premium or Data Center).
- Set required CI checks that must pass before a PR can be merged.

2 Configure Branch Restrictions

- Require all merges to integration-dev to go through a Pull Request.
- Require a minimum number of approvals before a PR can enter the queue.
- Enforce passing CI status checks as a merge prerequisite.
- Prevent direct pushes to integration-dev and preview branches.

3 Set Up Ephemeral Per-PR Preview Environments

- Configure your CI/CD pipeline (e.g. Bitbucket Pipelines) to auto-create a preview environment on PR open and auto-destroy it on PR close or merge.
- Each preview environment should be isolated — separate URL, separate data, no shared state.
- Post the preview URL as a comment on the PR automatically for easy access.
- Ensure the pipeline cleans up resources (containers, DNS records, cloud infra) on PR close.

4 Update Team Workflow & Communicate

• Document the new two-PR workflow: PR 1 to preview (test), PR 2 to integration-dev (integrate).

• Hold a team walkthrough session explaining the merge queue and ephemeral preview behaviour.

• Update any existing contributing guides, onboarding docs, or team wikis.

• Set expectations: PRs to integration-dev will not merge immediately — queue processing takes time.

5 Define Preview Environment Lifecycle Policy

• Specify who owns the preview environment configuration (e.g. DevOps / Platform team).

• Define the maximum lifetime of a preview environment (e.g. auto-destroy after 7 days of inactivity).

• Establish a naming convention for preview environments (e.g. preview-{PR-number}-{branch-slug}).

• Document the reset and teardown process and ensure it is automated, not manual.

SECTION 6

Process Comparison Summary

The table below provides a direct comparison of the three pipeline approaches across key engineering and process dimensions.

Merge conflict risk	**HIGH**	**MEDIUM**	**LOW**
Test isolation	**NO**	**PARTIAL**	**YES**
PR overhead per feature	1 PR	2 PRs	2 PRs + queue
Pipeline complexity	**LOW**	**MEDIUM**	**MEDIUM**
Race condition elimination	**NO**	**NO**	**YES**
Broken int-dev state risk	**HIGH**	**MEDIUM**	**LOW**
Developer visibility	**LOW**	**MEDIUM**	**YES**
Verdict	Current State	**Consider with mitigations**	**RECOMMENDED**

Summary Recommendation

- Adopt the two-stage pipeline to give developers isolated testing environments.
- Enable Bitbucket merge queues on integration-dev to eliminate race conditions.
- Make preview environments ephemeral and per-PR to avoid shared-branch conflicts.
- Communicate the updated workflow to the team with clear documentation and a walkthrough session.